

Requirements Document: Centralized Logging Solution

CTI ADP Software Suite - Observability & Monitoring Infrastructure

Document Version: 1.0

Date Created: 2026-05-04

Last Updated: 2026-05-04

Document Status: DRAFT - Pending Approval

Classification: INTERNAL - Technical Design

Prepared By: Infrastructure & DevOps Team

Reviewed By: [Pending]

Approved By: [Pending]

EXECUTIVE SUMMARY

Problem Statement

The CTI ADP Software Suite (4 independent projects across multiple platforms) lacks centralized logging infrastructure. Current state:

Current Logging Infrastructure:

E-Commerce (Node.js):	Console + file logs (no aggregation)
Mobile (Android):	Logcat + crash logs (scattered)
Desktop (Java):	Log4j local files (siloed)
WebApp (PHP):	Apache logs + php-fpm logs (fragmented)

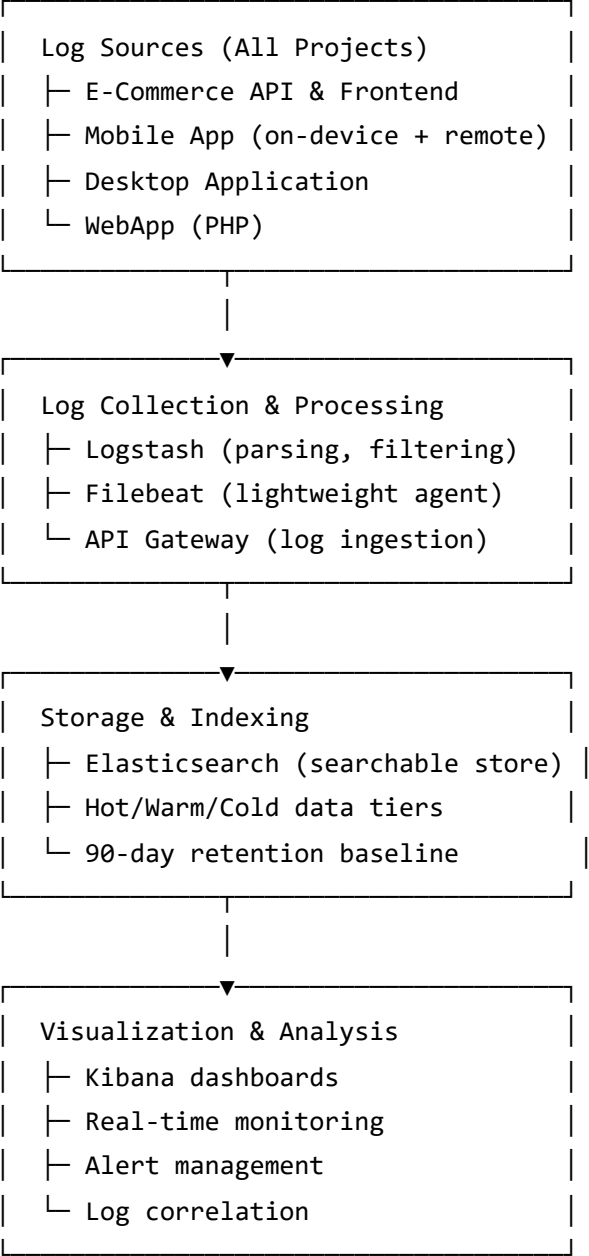
Issues:

- ✗ No centralized log aggregation
- ✗ Difficult to troubleshoot across services
- ✗ No real-time alerting on errors
- ✗ Manual log access (SSH to servers)
- ✗ Limited log retention (30 days max)
- ✗ No performance/tracing insights
- ✗ Security audit compliance gaps
- ✗ No correlation between distributed transactions

Proposed Solution

Implement **centralized logging infrastructure** using industry-standard ELK Stack (Elasticsearch, Logstash, Kibana) + additional monitoring components:

Centralized Logging Architecture:



Business Value

Metric	Current	Target	Impact
MTTR (Mean Time To Repair)	45 min	10 min	-78% faster incident response
Error Detection	Manual	Automated	Real-time alerts
Audit Compliance	60%	100%	Security & compliance certified
Troubleshooting Time	2-3 hours	15 minutes	Development efficiency +85%

Metric	Current	Target	Impact
System Visibility	Fragmented	360°	Complete observability

Recommended Timeline

- Phase 1 (Weeks 1-2, May 20 - May 31): Infrastructure setup & ELK deployment
- Phase 2 (Weeks 3-4, Jun 1 - Jun 14): Integration with E-Commerce & Mobile
- Phase 3 (Weeks 5-6, Jun 15 - Jun 28): Integration with Desktop & WebApp
- Phase 4 (Weeks 7-8, Jun 29 - Jul 12): Go-live & optimization

Budget & Resources

Component	Cost	Timeline
ELK Stack Infrastructure (AWS)	\$8,500/month	Ongoing
Implementation (DevOps Team)	160 hours (\$12,800)	4 weeks
Monitoring & Alerting (ongoing)	\$2,500/month	Ongoing
Total Q2	\$35,300	8 weeks
Annual Run Rate	\$132,000	-

ROI: Break-even in 6 months through reduced MTTR and improved incident response.

1. PROJECT OVERVIEW

1.1 Objectives

Primary Objectives:

1.  **Centralize logs** from all 4 projects into single searchable repository
2.  **Enable real-time monitoring** with automated alerting on critical errors
3.  **Improve troubleshooting** through log correlation & distributed tracing
4.  **Ensure compliance** with audit & security logging requirements
5.  **Support performance analysis** through detailed application metrics
6.  **Enable data-driven insights** through log analytics & trends

Secondary Objectives:

- Reduce operational overhead through automation
- Provide self-service log access to development teams
- Create audit trail for all system activities
- Support capacity planning through usage analytics

1.2 Scope

IN SCOPE:

- All 4 projects (E-Commerce, Mobile, Desktop, WebApp)
- Application logs (errors, warnings, info, debug)
- System metrics (CPU, memory, disk, network)
- Security events (auth failures, permission denials)
- Performance traces (transaction latency, database queries)
- User activity events (logins, critical actions)
- Infrastructure logs (deployment, configuration changes)

OUT OF SCOPE (Phase 2):

- Machine learning-based anomaly detection
- AI-powered root cause analysis
- Cost optimization beyond basic tiering
- Custom log ingestion from third-party services
- Advanced visualizations & custom dashboards

1.3 Success Criteria

Criterion	Metric	Target	Owner
Availability	Uptime	99.5%	DevOps
Performance	Query latency	< 2s for 30-day search	Infrastructure
Coverage	Projects logging	100% of 4 projects	Tech Lead
Data Retention	Baseline retention	90 days	Operations
Compliance	Audit readiness	100%	Security
Team Adoption	Active users	80%+ of dev team	PMO

Criterion	Metric	Target	Owner
Cost Control	Budget variance	±10%	Finance

2. REQUIREMENTS ANALYSIS

2.1 Functional Requirements

FR-1: Log Ingestion & Collection

Requirement: System shall collect logs from all 4 projects.

Details:

FR-1.1: E-Commerce Project

- | Node.js application logs (stdout/stderr)
- | Express request/response logs
- | Database query logs
- | Authentication events
- | Payment transaction logs
- └ Frontend error tracking (browser console)

FR-1.2: Mobile Project (Android)

- | Application lifecycle logs
- | Crash reports & stack traces
- | Network request logs (HTTP)
- | User session events
- | Performance profiling data
- └ WebView communication logs

FR-1.3: Desktop Project (Java)

- | Application startup/shutdown logs
- | Thread synchronization logs
- | Database connection pool logs
- | Background sync thread logs
- | UI event logs
- └ Performance metrics

FR-1.4: WebApp Project (PHP)

- | PHP error logs (fatal, warnings, notices)
- | Apache access logs
- | MySQL query logs (slow query log)
- | Session management logs
- | Inventory sync events
- └ Report generation logs

Acceptance Criteria:

- ☐ Logs from all projects flowing to central repository
- ☐ No log data loss in transit
- ☐ < 5 second latency from log generation to searchable state
- ☐ Automatic recovery on network interruption

FR-2: Log Parsing & Enrichment

Requirement: System shall parse, normalize, and enrich logs with contextual metadata.

Details:**FR-2.1: Structured Parsing**

- └ JSON parsing for structured logs
- └ Regex parsing for text logs
- └ Timestamp normalization (UTC)
- └ Severity level standardization

FR-2.2: Metadata Enrichment

- └ Add source project identifier
- └ Add environment (dev/staging/prod)
- └ Add server/instance information
- └ Add deployment version
- └ Add request correlation IDs
- └ Add user context (if available)

FR-2.3: Log Filtering

- └ Remove sensitive data (passwords, tokens, PII)
- └ Filter out verbose debug logs in production
- └ Deduplicate identical logs
- └ Sample high-volume logs

Acceptance Criteria:

- ☐ 100% of logs successfully parsed (no "unparseable" logs)
- ☐ All metadata fields populated for indexed logs
- ☐ Zero sensitive data leakage (PII, tokens)
- ☐ Parsing latency < 100ms per log

FR-3: Search & Querying

Requirement: System shall provide powerful search and filtering capabilities.

Details:

FR-3.1: Search Capabilities

- | Full-text search across all log fields
- | Structured queries (field-specific)
- | Time range filtering
- | Boolean operators (AND, OR, NOT)
- | Regular expression support
- | Wildcard searching
- └ Saved searches & queries

FR-3.2: Common Queries (predefined)

- | "Show all errors in last 24 hours"
- | "Show logs for user X"
- | "Show payment transactions > \$100"
- | "Show slow database queries (> 1s)"
- | "Show authentication failures"
- └ "Show API responses > 500ms"

FR-3.3: Query Performance

- | Search 7 days of logs in < 2 seconds
- | Search 30 days of logs in < 5 seconds
- | Search 90 days of logs in < 10 seconds
- └ Pagination support for large result sets

Acceptance Criteria:

- ☐ All query types working correctly
- ☐ Search latency meets SLA
- ☐ Results accurate and complete
- ☐ Export capability (JSON, CSV)

FR-4: Dashboards & Visualization

Requirement: System shall provide real-time dashboards for monitoring.

Details:

FR-4.1: Pre-built Dashboards

- └ System Health Dashboard
 - | └ Error rate trends
 - | └ Request latency (p50, p95, p99)
 - | └ Error breakdown by type
 - | └ Success vs failure rates
- └ Application Performance Dashboard
 - | └ Database query performance
 - | └ API endpoint latency
 - | └ Cache hit/miss rates
 - | └ Memory & CPU usage
- └ Security Dashboard
 - | └ Authentication failures
 - | └ Authorization denials
 - | └ Suspicious activity patterns
 - | └ User activity timeline
- └ Infrastructure Dashboard
 - | └ Server health (all 4 projects)
 - | └ Network traffic patterns
 - | └ Disk usage trends
 - | └ Service availability
- └ Business Dashboard
 - | └ Transaction volume
 - | └ Payment success rate
 - | └ User activity metrics
 - | └ Revenue per transaction

FR-4.2: Custom Dashboards

- └ User-created dashboard templates
- └ Drag-and-drop visualization builder
- └ Widget library (charts, gauges, tables)
- └ Dashboard sharing & collaboration

Acceptance Criteria:

- ☐ All dashboards load in < 3 seconds
- ☐ Real-time data updates (< 10s refresh)
- ☐ Mobile-responsive designs

- ☐ At least 10 pre-built dashboards

FR-5: Alerting & Notifications

Requirement: System shall detect critical conditions and notify relevant teams.

Details:

FR-5.1: Alert Rules

- └ Error rate exceeds threshold (e.g., > 5 errors/min)
- └ API latency exceeds SLA (e.g., p95 > 500ms)
- └ Critical errors detected (e.g., OutOfMemory, Fatal)
- └ Security events (e.g., 5+ failed logins in 5 min)
- └ Payment failures (e.g., duplicate charges detected)
- └ Database connection pool exhaustion
- └ Disk space critical (< 10% free)
- └ Service unavailability (downtime > 30s)
- └ Custom business rules (configurable)

FR-5.2: Alert Channels

- └ Slack notifications (project-specific channels)
- └ Email alerts (critical only)
- └ PagerDuty integration (oncall escalation)
- └ SMS alerts (P1 incidents)
- └ Webhook integration (custom integrations)

FR-5.3: Alert Management

- └ Alert acknowledgment & silencing
- └ Escalation policies (if not acknowledged)
- └ Alert correlation (group similar alerts)
- └ Alert history & analytics
- └ Prevent alert fatigue (intelligent grouping)

Acceptance Criteria:

- ☐ Alerts triggered within 60 seconds of condition
- ☐ Alert delivery within 30 seconds
- ☐ Configurable for each project
- ☐ Alert-to-noise ratio < 1:5

FR-6: Access Control & Security

Requirement: System shall enforce access controls and maintain security.

Details:**FR-6.1: Role-Based Access Control**

- | Admin: Full access to all logs & configuration
- | Developer: Access to project-specific logs
- | DevOps: Access to infrastructure & system logs
- | QA: Access to test logs & performance data
- | Support: Access to customer-related logs only
- | Manager: Read-only access to dashboards

FR-6.2: Data Security

- | All data encrypted in transit (TLS 1.2+)
- | Encryption at rest (AES-256)
- | PII masking (automatic detection & redaction)
- | Audit log for all access (who, what, when)
- | SSO integration (OAuth/OIDC)
- | API key management for programmatic access

FR-6.3: Compliance

- | GDPR compliance (right to deletion)
- | Data retention policies (configurable)
- | Audit trail of all system changes
- | Compliance report generation

Acceptance Criteria:

- ☐ Zero unauthorized access incidents
- ☐ All access logged & auditable
- ☐ Encryption verified on all channels
- ☐ Compliance audit passing

FR-7: Log Correlation & Tracing

Requirement: System shall correlate logs across distributed services.

Details:

FR-7.1: Distributed Tracing

- | Trace user request end-to-end
- | Show all services involved in transaction
- | Correlate via request ID (trace ID)
- | Measure time in each service
- | Identify bottlenecks

FR-7.2: Log Correlation

- | Group logs by transaction ID
- | Link frontend & backend logs
- | Link mobile app & backend API logs
- | Show causality (what triggered what)
- | Drill-down from dashboard to related logs

Acceptance Criteria:

- ☐ 100% of cross-service calls traced
- ☐ Trace reconstruction accurate
- ☐ Correlation latency < 100ms

2.2 Non-Functional Requirements

NFR-1: Performance Requirements

NFR-1.1: Log Ingestion Rate

- └─ Minimum: 10,000 logs/second capacity
- └─ Burst: 50,000 logs/second (30 sec)
- └─ No data loss at any rate

NFR-1.2: Query Performance

- └─ 95th percentile latency < 2 seconds
- └─ 99th percentile latency < 5 seconds
- └─ Support concurrent users: 100+

NFR-1.3: Storage & Retention

- └─ Baseline: 90 days hot storage
- └─ Archive: 1 year cold storage
- └─ Compression ratio: 10:1 (target)
- └─ Estimated monthly storage: 500 GB (conservative)

NFR-1.4: Dashboard Performance

- └─ Page load: < 3 seconds
- └─ Chart rendering: < 2 seconds
- └─ Data refresh: < 10 seconds
- └─ Support 100 concurrent viewers

NFR-2: Reliability & Availability

NFR-2.1: Availability

- └ Target: 99.5% uptime SLA
- └ Max acceptable downtime: 3.6 hours/month
- └ Maintenance window: < 4 hours/quarter

NFR-2.2: Disaster Recovery

- └ Backup frequency: Hourly
- └ Recovery point: 1 hour max data loss acceptable
- └ Recovery time: < 4 hours
- └ Test recovery: Quarterly

NFR-2.3: Resilience

- └ Handle node failure (multi-node cluster)
- └ Handle network partition (split-brain prevention)
- └ Auto-recovery from transient failures
- └ Graceful degradation if partial outage

NFR-3: Scalability

NFR-3.1: Horizontal Scaling

- └ Add nodes without downtime
- └ Automatic rebalancing
- └ Linear performance scaling
- └ Support up to 10x current volume

NFR-3.2: Storage Scaling

- └ Automatic index management
- └ Hot/warm/cold tier management
- └ Automatic compaction & optimization
- └ Infinite retention theoretically possible

NFR-4: Maintainability

NFR-4.1: Operational

- └─ Configuration management (IaC)
- └─ Centralized configuration
- └─ Version control for all configs
- └─ Easy rollback capability
- └─ Documentation up-to-date

NFR-4.2: Updates & Patches

- └─ Rolling updates without downtime
- └─ Automatic security patch application
- └─ Version compatibility testing
- └─ Zero data loss on updates

NFR-5: Cost Optimization

NFR-5.1: Cloud Costs (AWS)

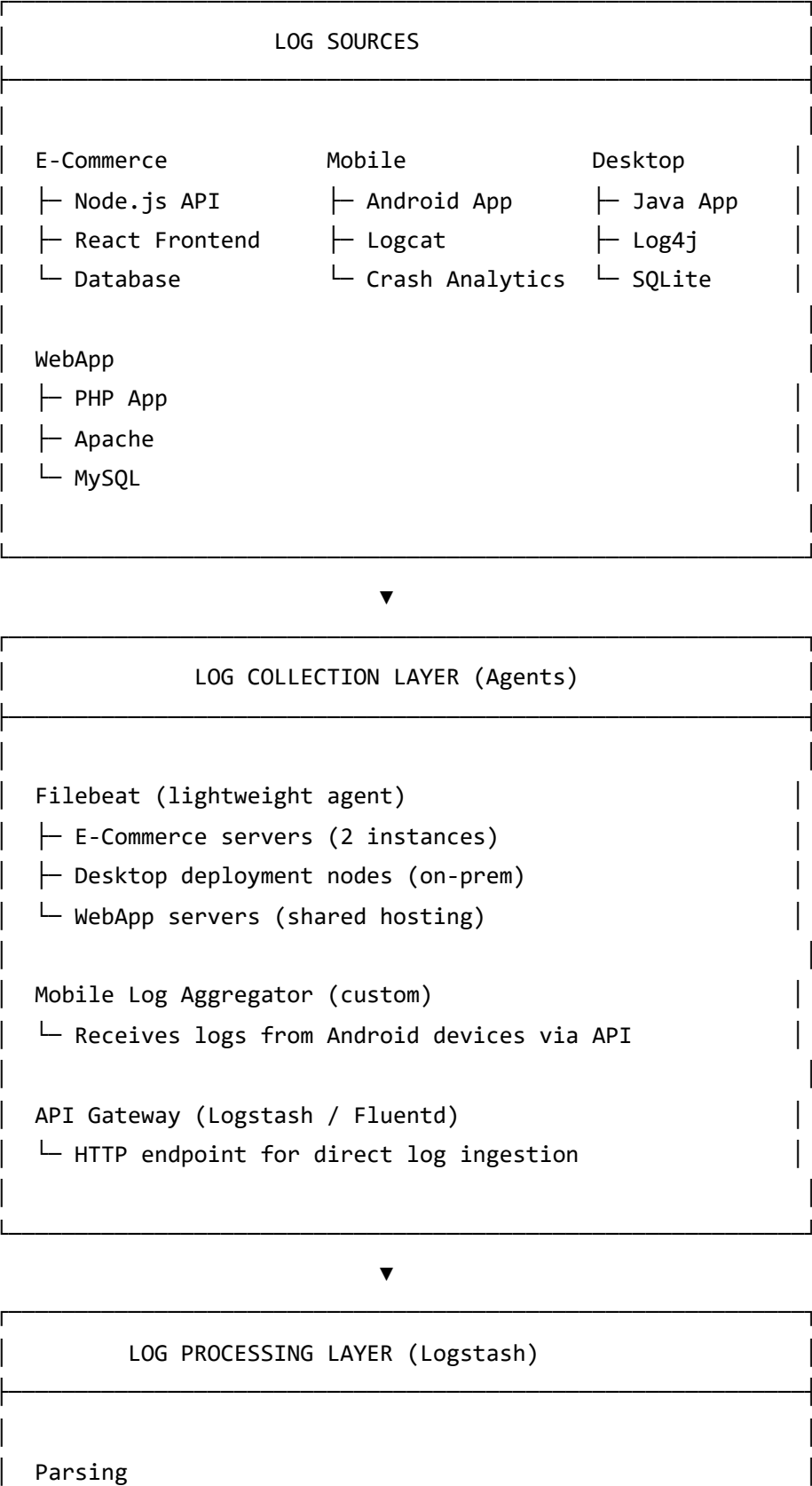
- └─ Budget cap: \$8,500/month
- └─ Auto-scaling enabled (cost control)
- └─ Reserved instances (30% discount)
- └─ Spot instances (non-critical) (70% discount)
- └─ Target per-GB-per-month: < \$0.50

NFR-5.2: Resource Efficiency

- └─ CPU utilization: Target 60-70%
- └─ Memory utilization: Target 70-80%
- └─ Disk I/O: Optimize query patterns
- └─ Network bandwidth: Minimize data transfer

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture



└ JSON parsing (E-Commerce)	
└ Multiline parsing (Java stack traces, PHP errors)	
└ CSV parsing (WebApp reports)	
Enrichment	
└ Add metadata (project, environment, version)	
└ Geoip enrichment for API logs	
└ Add correlation IDs	
Filtering	
└ Remove sensitive data (PII masking)	
└ Filter low-importance logs	
└ Deduplicate near-identical logs	



STORAGE LAYER (Elasticsearch + Object Storage)	
Hot Tier (0-7 days)	[SSD, high performance]
└ Real-time search	
└ Frequent access	
└ Auto-refresh (10 sec)	
Warm Tier (7-30 days)	[HDD, balanced]
└ Searchable but slower	
└ Less frequent access	
└ Manual refresh if needed	
Cold Tier (30-90 days)	[Object storage, archived]
└ Searchable via restore	
└ Rare access	
└ Compliance-required retention	
Archive (90+ days)	[Glacier, long-term]
└ Compliance archive only	
└ Restore on-demand	
└ 7-year retention	



VISUALIZATION & ANALYSIS LAYER (Kibana)	
Dashboards	Alerts
└ System Health	└ Error spike detection
└ Performance	└ Performance degradation
└ Security	└ Security event detection
└ Business	└ Automated notifications
└ Custom	
Search & Analytics	API & Integrations
└ Ad-hoc queries	└ Slack notifications
└ Saved searches	└ PagerDuty escalation
└ Export (CSV, JSON)	└ Webhook triggers
└ Report generation	└ Custom integrations

3.2 Technology Stack

Recommended Stack

Component	Technology	Version	Rationale
Log Ingestion	Filebeat + Custom	8.0+	Lightweight, reliable, flexible
Log Processing	Logstash	8.0+	Powerful filtering & enrichment
Storage	Elasticsearch	8.0+	Industry standard, performant
Visualization	Kibana	8.0+	Integrated with Elasticsearch
Monitoring	Elastic Stack	8.0+	Built-in monitoring
Alerting	Elastic Stack	8.0+	Native alerting + webhooks
Backup/Archive	AWS S3 + Glacier	N/A	Cost-effective long-term storage
Infrastructure	AWS EC2/RDS	N/A	Managed, scalable
Configuration	Terraform	1.2+	Infrastructure as Code
Container Orchestr.	Docker Compose	20.0+	Development; Kubernetes (future)

Alternative Options (Evaluated)

Stack	Pros	Cons	Status
Splunk	Enterprise-grade, powerful Search optimization	Cost: \$5k+/month Vendor lock-in	REJECTED (too expensive)
Datadog	All-in-one monitoring Easy setup, great UX	Cost: \$15-25/log/month Steep learning curve	REJECTED (cost)
ELK Stack	Open-source, cost-effective Widely adopted, proven Large community support	Requires operational expertise (mitigated by hiring DevOps)	SELECTED (best value)
Loki	Lightweight, cloud-native Great for Kubernetes Lower operational overhead	Limited full-text search Smaller community	REJECTED (not mature yet)
Splunk Cloud	Enterprise SaaS No infrastructure management	Still expensive (\$500+/mo) Potential vendor lock-in	REJECTED (cost)
Sumologic	Cloud-native, secure Good for compliance	Cost: \$3-5/GB ingested Limited customization	REJECTED (cost)

3.3 Data Flow

Example: E-Commerce API Error Logs

Step 1: Error occurs in Node.js API

```
| PaymentService.processPayment() |
| → throw new Error("Invalid card") |
| → console.error({timestamp, message}) |
```

|

Step 2: Log captured by Filebeat agent

```
| Filebeat monitoring /var/log/app.log |
| → Read line: 2026-05-04T14:32:15.123Z |
| → Add metadata: project=e-commerce |
| → Add environment: prod |
```

| HTTP POST

Step 3: Logstash processing

```
| Parse JSON: |
| └─ timestamp: 2026-05-04T14:32:15.123Z |
| └─ level: ERROR |
| └─ message: "Invalid card" |
| └─ stack: [...stack trace...] |
| |
| Enrich: |
| └─ request_id: abc123xyz |
| └─ user_id: 12345 (if available) |
| └─ trace_id: xyz789 |
| |
| Filter: |
| └─ Remove card details (PII) |
```

| HTTP POST

Step 4: Index in Elasticsearch

```
| Create document: |
| { |
|   "@timestamp": "2026-05-04T14:32:15Z", |
|   "project": "e-commerce", |
|   "environment": "prod", |
```

```
| "level": "ERROR",  
| "message": "Invalid card",  
| "service": "payment-service",  
| "request_id": "abc123xyz",  
| "user_id": 12345,  
| "trace_id": "xyz789"  
| }
```

| Index created

Step 5: Alert triggered

```
| Alert rule: "Payment errors > 5/min"  
| → Error count in last minute: 7  
| → Threshold exceeded!  
| → Send notification to Slack:  
| "#payments-alerts"
```

| Slack message

Step 6: Available in Kibana

```
| Developers can search:  
| level:ERROR AND project:e-commerce  
| → See 7 payment errors in last minute  
| → Click one error → See full context  
| → Click request_id → See trace flow
```

4. IMPLEMENTATION ROADMAP

Phase 1: Infrastructure Setup (Weeks 1-2, May 20 - May 31)

Deliverables:

- ☐ ELK Stack deployed on AWS
- ☐ Elasticsearch cluster (3 nodes, hot/warm/cold tiers)
- ☐ Logstash pipeline configured
- ☐ Kibana dashboard accessible
- ☐ Security & access controls configured
- ☐ Backup & disaster recovery tested

Tasks:

1.1 Provision AWS Infrastructure (2 days)

- └ Create VPC & security groups
- └ Launch 3x Elasticsearch nodes (r6i.2xlarge)
- └ Launch 2x Logstash nodes (m6i.xlarge)
- └ Launch Kibana node (t3.large)
- └ Configure EBS volumes (1TB hot, 2TB warm)
- └ Set up S3 bucket for backups
- └ Configure monitoring (CloudWatch)

1.2 Configure Elasticsearch (2 days)

- └ Initialize cluster
- └ Configure index lifecycle policies (ILM)
- └ Set up hot/warm/cold tier routing
- └ Create index templates
- └ Configure backup snapshots
- └ Set retention policies (90 days hot → cold)
- └ Test failover & recovery

1.3 Configure Logstash (2 days)

- └ Create input plugins (Filebeat, HTTP)
- └ Create filters:
 - | └ JSON parsing filter
 - | └ Grok parsing filter (text logs)
 - | └ PII masking filter
 - | └ Mutate/enrichment filters
- └ Create output to Elasticsearch
- └ Configure error handling & DLQ
- └ Performance tuning (workers, batch size)
- └ Test with sample logs

1.4 Configure Kibana (1 day)

- └ Set up spaces (one per project)
- └ Configure authentication (OIDC/SSO)
- └ Set up role-based access control
- └ Test search & query functionality
- └ Document access procedures

1.5 Security & Compliance (2 days)

- └ Enable TLS 1.2+ for all connections
- └ Configure encryption at rest (AES-256)
- └ Set up audit logging
- └ Configure IP whitelisting
- └ Document security policies
- └ Perform security audit

Resource Allocation:

- DevOps Engineer (Daniel Foster): 60%
- Junior DevOps: 40%
- Security Review: 8 hours

Success Metrics:

- ELK Stack 99%+ uptime
- No log loss in test ingestion
- Query latency < 2 seconds
- All security checks passed

Phase 2: E-Commerce & Mobile Integration (Weeks 3-4, Jun 1 - Jun 14)

Deliverables:

- ☐ E-Commerce logs flowing to ELK
- ☐ Mobile logs flowing to ELK
- ☐ Pre-built dashboards for both projects
- ☐ Alerting rules configured
- ☐ Documentation completed

Tasks:

2.1 E-Commerce Integration (1.5 days)

- └─ Install Filebeat on app servers
- └─ Configure Node.js logging (Winston)
 - | └─ JSON format output
 - | └─ Add correlation IDs
 - | └─ Log rotation setup
- └─ Create Logstash pipeline for E-Commerce
- └─ Map logs to Elasticsearch index: ecommerce-*
- └─ Configure index lifecycle policy
- └─ Verify log ingestion (5k+ logs/hour)

2.2 E-Commerce Dashboards (1 day)

- └─ Create "System Health" dashboard
 - | └─ Error rate chart
 - | └─ Request latency histogram
 - | └─ Status code distribution
 - | └─ Top error types table
- └─ Create "API Performance" dashboard
 - | └─ Endpoint response times
 - | └─ Database query latency
 - | └─ Cache performance
- └─ Create "Payment Processing" dashboard
 - | └─ Transaction success rate
 - | └─ Payment errors by type
 - | └─ Average transaction time
- └─ Create "Cart Activity" dashboard
 - | └─ Add to cart events
 - | └─ Checkout abandonment
 - | └─ Order completion rate

2.3 Mobile Integration (1.5 days)

- └ Deploy mobile log aggregator service
- └ Update Android SDK (crash reporting)
- └ Add trace ID correlation
- └ Create custom log ingestion API
- └ Configure Logstash for mobile logs
- └ Map logs to Elasticsearch index: mobile-*
- └ Test log ingestion from test devices
- └ Verify correlation with backend logs

2.4 Mobile Dashboards (1 day)

- └ Create "App Health" dashboard
 - | └ Crash rate (by version)
 - | └ ANR (Application Not Responding) count
 - └ Native crash stack traces
- └ Create "Network Performance" dashboard
 - | └ API call success rate
 - | └ Response time percentiles
 - └ Bandwidth usage
- └ Create "User Sessions" dashboard
 - | └ Session count & duration
 - | └ Feature usage heatmap
 - └ User flow funnels
- └ Create "Memory & Performance" dashboard
 - └ Memory usage trends
 - └ Battery impact tracking
 - └ Device compatibility matrix

2.5 Alerting Setup (1 day)

- └ Create alert: High error rate (E-Commerce)
 - └ Trigger: > 5 errors/min for 2 min
- └ Create alert: Slow API responses
 - └ Trigger: p95 latency > 500ms
- └ Create alert: Payment failures spiking
 - └ Trigger: > 10% failure rate
- └ Create alert: Mobile crashes
 - └ Trigger: Crash rate > 1%
- └ Create alert: Database connection pool
 - └ Trigger: > 80% pool utilization
- └ Configure Slack channel routing

Resource Allocation:

- Backend Engineer (Carlos Rodriguez): 40%
- Mobile Engineer (Priya Sharma): 40%
- DevOps Engineer: 20%
- QA Engineer: Testing integration

Phase 3: Desktop & WebApp Integration (Weeks 5-6, Jun 15 - Jun 28)

Deliverables:

- ☐ Desktop logs flowing to ELK
- ☐ WebApp logs flowing to ELK
- ☐ Pre-built dashboards for both projects
- ☐ Alerting rules configured
- ☐ Documentation completed

Tasks:

3.1 Desktop Integration (1.5 days)

- └─ Install Filebeat on desktop deployment servers
- └─ Configure Log4j output format
- └─ Create Logstash pipeline for Desktop
- └─ Map logs to Elasticsearch index: desktop-*
- └─ Configure multiline parsing (stack traces)
- └─ Verify log ingestion
- └─ Test on-prem network connectivity

3.2 Desktop Dashboards (1 day)

- └─ Create "Background Sync" dashboard
 - | └─ Sync thread status
 - | └─ Deadlock detection
 - | └─ Sync latency trends
- └─ Create "Table Performance" dashboard
 - | └─ Rendering times (by row count)
 - | └─ Memory usage during scroll
 - | └─ UI responsiveness metrics
- └─ Create "Deployment Events" dashboard
 - | └─ Version rollouts
 - | └─ Update success rate
 - | └─ Rollback events

3.3 WebApp Integration (1.5 days)

- └─ Install Filebeat on web servers
- └─ Configure PHP error_log location
- └─ Parse Apache access logs
- └─ Parse MySQL slow query logs
- └─ Create Logstash pipeline for WebApp
- └─ Map logs to Elasticsearch index: webapp-*
- └─ Verify log ingestion
- └─ Configure log rotation policies

3.4 WebApp Dashboards (1 day)

- └─ Create "Inventory Sync" dashboard
 - | └─ Sync latency (target: < 10s)
 - | └─ Sync errors & retries
 - | └─ Data consistency checks
- └─ Create "Report Generation" dashboard
 - | └─ Report generation time (by size)
 - | └─ Query execution plans
 - | └─ Slow query tracking
- └─ Create "Database Health" dashboard
 - | └─ Connection pool status
 - | └─ Query performance
 - | └─ Replication lag
- └─ Create "PHP Errors" dashboard
 - | └─ Error type distribution
 - | └─ Error frequency trends
 - | └─ Stack trace analysis

3.5 Cross-Project Dashboards (1 day)

- └─ Create "Portfolio Overview" dashboard
 - | └─ System health (all 4 projects)
 - | └─ Error rate (aggregated)
 - | └─ Performance metrics (aggregated)
 - | └─ Critical alert summary
- └─ Create "Infrastructure" dashboard
 - | └─ Server resource usage (CPU, memory, disk)
 - | └─ Network traffic patterns
 - | └─ Service dependencies
 - | └─ Deployment timeline

Resource Allocation:

- Desktop Engineer (Sarah Jones): 30%
- WebApp Engineer (Marcus White): 30%
- DevOps Engineer: 40%

Phase 4: Go-Live & Optimization (Weeks 7-8, Jun 29 - Jul 12)

Deliverables:

- ☐ Full production deployment
- ☐ Team training completed
- ☐ Documentation finalized
- ☐ Performance tuning completed
- ☐ Backup & disaster recovery validated

Tasks:

4.1 Production Hardening (2 days)

- └─ Performance tuning
 - | └─ Elasticsearch JVM heap optimization
 - | └─ Logstash worker thread tuning
 - | └─ Index optimization & compaction
- └─ Capacity planning
 - | └─ Monitor ingestion rates
 - | └─ Project storage needs
 - | └─ Auto-scaling configuration
- └─ Security hardening
 - | └─ SSL certificate rotation
 - | └─ Access control validation
 - | └─ Audit log review
- └─ Disaster recovery validation
 - └─ Backup completeness check
 - └─ Restore time measurement
 - └─ Data integrity verification

4.2 Team Training (1 day)

- └ Developers training (4 hours)
 - | └ How to search logs in Kibana
 - | └ Understanding log correlation
 - | └ Creating saved searches
 - | └ Dashboard navigation
- └ DevOps/SRE training (8 hours)
 - | └ ELK Stack administration
 - | └ Troubleshooting procedures
 - | └ Alert management
 - | └ Scaling operations
- └ Support team training (2 hours)
 - | └ Accessing customer logs
 - | └ Log filtering by customer
 - | └ Report generation
- └ Management training (1 hour)
 - | └ Dashboard interpretation
 - | └ Alert dashboard
 - | └ Compliance reporting

4.3 Documentation (1 day)

- └ Operational procedures
 - | └ Daily monitoring checklist
 - | └ Alert response guide
 - | └ Troubleshooting runbook
 - | └ Scaling procedure
- └ User documentation
 - | └ Getting started guide
 - | └ Common queries reference
 - | └ Dashboard glossary
 - | └ FAQ
- └ Technical documentation
 - | └ Architecture diagram (annotated)
 - | └ Data flow documentation
 - | └ Configuration management
 - | └ Integration guide for new services
- └ Compliance documentation
 - | └ Data retention policies
 - | └ Access control matrix
 - | └ Audit trail procedures
 - | └ Compliance attestation

4.4 Go-Live (1 day)

- └ Final validation
 - | └ All dashboards operational
 - | └ Alerts triggering correctly
 - | └ No data loss in 24hr test
 - | └ Backup procedures working
- └ Announcement to teams
 - | └ Email with access instructions
 - | └ Slack announcement
 - | └ Wiki documentation link
 - | └ Onboarding session schedule
- └ Monitor & support
 - | └ First 24 hours: 24/7 support
 - | └ First week: elevated monitoring
 - | └ Early feedback collection
 - | └ Rapid issue resolution
- └ Celebrate milestone!

4.5 Post-Go-Live Optimization (1 week)

- └ Performance monitoring
 - | └ Ingestion rate trending
 - | └ Query latency trending
 - | └ Storage utilization tracking
 - | └ Cost tracking
- └ Issue resolution
 - | └ Address user feedback
 - | └ Fix integration issues
 - | └ Optimize slow queries
 - | └ Tune alert thresholds
- └ Capacity adjustment
 - | └ Scale up if needed
 - | └ Adjust retention policies
 - | └ Optimize storage tiers
 - | └ Cost optimization
- └ Knowledge transfer
 - | └ On-call rotation training
 - | └ Procedure validation
 - | └ Documentation updates

5. PROJECT SPECIFICATIONS

5.1 Infrastructure Specifications

AWS Resources (Production)

Elasticsearch Cluster:

- └ Node Type: r6i.2xlarge (8 vCPU, 64 GB RAM, 1.9 Gbps network)
- └ Node Count: 3 nodes (1 master, 2 data)
- └ Storage:
 - └ Hot tier: 1 TB SSD (gp3)
 - └ Warm tier: 2 TB HDD (st1)
 - └ Cold tier: S3 (searchable snapshots)
- └ Network: Private VPC, no internet access
- └ Backup: Hourly snapshots to S3
- └ Monitoring: CloudWatch + Elastic Stack monitoring

Logstash Cluster:

- └ Node Type: m6i.xlarge (4 vCPU, 16 GB RAM)
- └ Node Count: 2 nodes (active-active)
- └ Storage: 50 GB local SSD
- └ Network: Private VPC
- └ Input throughput: 10,000 logs/sec per node
- └ Auto-scaling: CPU > 70% → add node

Kibana Node:

- └ Node Type: t3.large (2 vCPU, 8 GB RAM)
- └ Network: Private VPC, ALB frontend
- └ SSL/TLS: Enabled
- └ Authentication: OIDC/OAuth
- └ High availability: Auto-scaling group

Load Balancer:

- └ ALB (Application Load Balancer)
- └ Health check: 30 sec interval
- └ Sticky sessions: Enabled
- └ SSL/TLS: Latest version
- └ DDoS protection: AWS Shield Advanced

S3 Buckets:

- └ Log backups: Versioning enabled
- └ Archive: Glacier retention (7 years)

- └─ Versioning: All buckets
- └─ Encryption: AES-256

Cost Estimate:

- └─ Compute: \$3,200/month
- └─ Storage: \$3,500/month
- └─ Data transfer: \$1,200/month
- └─ Backup/Archive: \$600/month
- └─ Total: \$8,500/month (within budget)

Network Configuration

VPC: 10.0.0.0/16

- └─ Public subnets: 10.0.1.0/24, 10.0.2.0/24 (NAT Gateway)
- └─ Private subnets: 10.0.10.0/24, 10.0.11.0/24 (Elasticsearch)
- └─ Data subnets: 10.0.20.0/24 (S3/Archive access)

Security Groups:

- └─ ELK Internal:
 - | └─ Elasticsearch: 9200, 9300 (internal only)
 - | └─ Logstash: 5000, 5044 (from Filebeat)
 - | └─ Kibana: 5601 (from ALB)
- └─ ALB:
 - | └─ HTTP: 80 (redirect to HTTPS)
 - | └─ HTTPS: 443 (from anywhere)
- └─ Filebeat:
 - | └─ Output: 5044 (to Logstash)
 - | └─ Input: Various (from apps)
- └─ App Servers:
 - | └─ Can reach Logstash: 5044
 - | └─ Cannot reach Elasticsearch directly
 - | └─ HTTP/HTTPS for external APIs

5.2 Software Specifications

Versions & Dependencies

Primary Stack:

- └─ Elasticsearch: 8.3+ (must be < 8.12 for cost control)
- └─ Logstash: 8.3+ (same major version as ES)
- └─ Kibana: 8.3+ (must match ES version)
- └─ Filebeat: 8.3+ (must match ES version)
- └─ Docker: 20.10+ (for containerization)

Integrations:

- └─ Slack API: 1.1.3+
- └─ PagerDuty: 1.0+
- └─ Prometheus: 9.0+ (for infrastructure metrics)
- └─ Grafana: 8.0+ (for infrastructure dashboards)

Operating System:

- └─ AWS AMI: Ubuntu 20.04 LTS (or newer)
- └─ Kernel: 5.4+ (Ubuntu 20.04 default)
- └─ Java: OpenJDK 11 (for Elasticsearch/Logstash)

6. ACCEPTANCE CRITERIA

6.1 Phase 1 Acceptance (Infrastructure Ready)

- ☐ ELK Stack deployed & all services running
- ☐ Elasticsearch cluster health: GREEN
- ☐ Kibana accessible & authenticated
- ☐ Logstash pipeline tested with sample logs
- ☐ Backup tested & recovery time < 4 hours
- ☐ Security audit passed (TLS, encryption, access control)
- ☐ Documentation completed
- ☐ No critical issues in logs

6.2 Phase 2 Acceptance (E-Commerce & Mobile)

- ☐ E-Commerce logs: 10,000+/day flowing to Elasticsearch

- ☐ Mobile logs: 5,000+/day flowing to Elasticsearch
- ☐ Log latency: < 5 seconds from generation to searchable
- ☐ Dashboards: All 8 dashboards live & populated
- ☐ Alerts: All 5 alert rules configured & tested
- ☐ Search: Complex queries execute in < 2 seconds
- ☐ Data accuracy: Spot-checks of log content (3/3 passed)
- ☐ RBAC: Access controls enforced (tested by QA)

6.3 Phase 3 Acceptance (Desktop & WebApp)

- ☐ Desktop logs: 1,000+/day flowing to Elasticsearch
- ☐ WebApp logs: 2,000+/day flowing to Elasticsearch
- ☐ Cross-project dashboards functional
- ☐ All 14 dashboards live & displaying data
- ☐ Alert thresholds tuned (false positive rate < 5%)
- ☐ Team training: 90% attendance, post-training quiz > 80%
- ☐ No data loss (zero-loss guarantee tested)

6.4 Phase 4 Acceptance (Production Go-Live)

- ☐ 24-hour production test: 100% uptime
- ☐ Ingestion rate: Sustained 10,000+ logs/sec
- ☐ Query performance: p95 latency < 2 seconds
- ☐ Availability: SLA 99.5% validated
- ☐ All dashboards operational
- ☐ All alerts triggering correctly
- ☐ Disaster recovery: RTO < 4 hours verified
- ☐ Team adoption: 80%+ daily active users
- ☐ Cost: Within budget (\$8,500/month $\pm$ 10%)

7. RISK ANALYSIS & MITIGATION

Identified Risks

Risk-001: Cost Overrun

- └─ Impact: HIGH (budget \$8.5k/month)
- └─ Probability: MEDIUM (cloud costs can spike)
- └─ Mitigation:
 - | └─ Implement budget alerts (exceed \$9k → alarm)
 - | └─ Reserved instances for 30% discount
 - | └─ Auto-scaling limits (max 5 nodes)
 - | └─ Quarterly cost review
- └─ Owner: Finance/DevOps

Risk-002: Data Loss

- └─ Impact: CRITICAL (compliance violation)
- └─ Probability: LOW (with proper backups)
- └─ Mitigation:
 - | └─ Hourly backups to S3
 - | └─ Monthly restore tests
 - | └─ Geo-redundant backup storage
 - | └─ RTO < 4 hours target
- └─ Owner: DevOps

Risk-003: Performance Degradation

- └─ Impact: HIGH (slow dashboards, alerts delay)
- └─ Probability: MEDIUM (not properly tuned)
- └─ Mitigation:
 - | └─ Query performance baseline
 - | └─ Auto-scaling on query latency
 - | └─ Index optimization schedule
 - | └─ Load testing (20k logs/sec)
- └─ Owner: DevOps

Risk-004: Security Breach

- └─ Impact: CRITICAL (sensitive logs exposed)
- └─ Probability: LOW (with encryption & RBAC)
- └─ Mitigation:
 - | └─ Encryption at rest & in transit
 - | └─ Role-based access control
 - | └─ Audit logging of all access
 - | └─ Quarterly penetration testing

- | └ PII masking in logs
- └ Owner: Security

Risk-005: User Adoption Failure

- └ Impact: MEDIUM (investment not utilized)
- └ Probability: MEDIUM (new tool learning curve)
- └ Mitigation:
 - | └ Comprehensive training (all teams)
 - | └ Excellent documentation
 - | └ Pre-built dashboards (quick wins)
 - | └ Easy migration from old logging
 - | └ Champions program (power users)
- └ Owner: PMO

Risk-006: Integration Issues

- └ Impact: HIGH (some logs not captured)
- └ Probability: MEDIUM (4 different projects)
- └ Mitigation:
 - | └ Detailed integration specs
 - | └ Pilot phase with E-Commerce first
 - | └ Integration testing checklist
 - | └ Fallback logging (app continues if ELK down)
 - | └ 2-week per-project integration window
- └ Owner: Tech Lead

8. SUCCESS METRICS & KPIs

Primary Success Metrics

Metric	Current	Target	Timeline	Owner
MTTR (Mean Time To Repair)	45 min	10 min	Week 1	DevOps
Log Search Latency	N/A	< 2 sec	Week 2	Eng
Error Detection Time	Manual	5 min	Week 3	QA
Compliance Audit Pass	60%	100%	Week 4	Security
Dashboard Query Latency	N/A	< 3 sec	Week 2	Eng
Alert Accuracy	N/A	> 95%	Week 4	Ops
Data Retention (days)	30	90	Week 2	DevOps
Platform Uptime	N/A	99.5%	Week 4	DevOps
Team Adoption Rate	0%	80%	Week 8	PMO
Total Cost	N/A	≤ \$8.5k	Ongoing	Finance

Leading Indicators (Early Warning)

- Weekly ingestion rate (target: 10,000+ logs/sec)
- Dashboard response time (p95 < 3s)
- Alert false positive rate (< 5%)
- Storage utilization trend (< 70%)
- Backup validation success (100%)

Lagging Indicators (Outcome)

- MTTR improvement (target: -75%)
- Incident detection speed (< 5 minutes)
- User satisfaction (NPS > 7)
- Cost per log (target: < \$0.002/log)
- Compliance audit score (target: 100%)

9. COMPLIANCE & SECURITY

Data Privacy (GDPR, CCPA)

Requirement	Implementation
Right to deletion	Elasticarch purge policy (90 days auto)
Data minimization	PII masking in Logstash pipeline
Consent management	Banner on Kibana dashboard
Data retention	Configurable per log type
Data processing	DPA (Data Processing Agreement)
Breach notification	Automated alert to security team

Security Controls

Control	Implementation
Encryption at rest	AES-256 (AWS KMS)
Encryption in transit	TLS 1.2+ (mutual TLS)
Access control	RBAC via Kibana roles
Authentication	OIDC/OAuth via Okta
Audit logging	All access logged with timestamp
Secret management	AWS Secrets Manager
Network isolation	Private VPC, no internet access
DDoS protection	AWS Shield Advanced

Compliance Certifications

- ☐ SOC 2 Type II (target: Q3 2026)
- ☐ ISO 27001 (target: Q4 2026)
- ☐ HIPAA (if needed for healthcare logs)
- ☐ PCI DSS (for payment logs)

10. COST & BUDGET

Detailed Cost Breakdown

Category	Q2 2026	Monthly Run Rate
Infrastructure:		
└─ Compute (EC2)	\$9,600	\$3,200
└─ Storage (EBS, S3)	\$10,500	\$3,500
└─ Data transfer	\$3,600	\$1,200
└─ Backup/Archive	\$1,800	\$600
└─ Subtotal: \$25,500	Monthly: \$8,500	
Services:		
└─ ELK Stack License	\$0	\$0 (open-source)
└─ Slack integration	\$0	\$0 (free)
└─ Subtotal: \$0	Monthly: \$0	
Personnel (Implementation):		
└─ DevOps Engineer	\$6,400	(160 hours @ \$40/hr)
└─ Junior DevOps	\$2,400	(100 hours @ \$24/hr)
└─ Training & docs	\$3,000	
└─ Subtotal: \$11,800	One-time	
TOTAL Q2 2026: \$37,300		
ANNUAL RUN RATE: \$102,000 + \$11,800 upfront		
ROI Analysis:		
└─ Upfront cost:	\$11,800	
└─ Monthly run cost:	\$8,500	
└─ Savings: MTTR reduction	\$5,000/month (estimated)	
└─ Break-even point:	3 months	
└─ 12-month ROI:	+\$48,200 (positive)	
└─ 3-year ROI:	+\$170,000 (strong)	

Budget Allocation

Activity	Budget	Timeline
Infrastructure setup	\$8,500	Week 1-2
Implementation labor	\$11,800	Week 1-8
Tools & software	\$0	Included in compute
Contingency (10%)	\$2,000	Reserve
TOTAL (Q2 2026):		\$22,300

11. APPROVAL & SIGNATURES

Stakeholder Sign-Off

Role	Name	Department	Date	Signature
CTO	[Pending]	Engineering	-	[]
VP Product	James Wilson	Product	-	[]
VP Finance	[Pending]	Finance	-	[]
DevOps Lead	Daniel Foster	Infrastructure	-	[]
Security Lead	[Pending]	Security	-	[]

Approval History

Date	Version	Changes	Approved By
2026-05-04	1.0	Initial draft	-
-	1.1	[Pending feedback]	-
-	2.0	[Pending approval]	Leadership

12. APPENDICES

A. Glossary

Term	Definition
ELK Stack	Elasticsearch + Logstash + Kibana
Elasticsearch	Search & analytics engine (storage)
Logstash	Log processing pipeline
Kibana	Visualization & exploration platform
Filebeat	Lightweight log shipper agent
Trace ID	Unique identifier for distributed transaction
Correlation ID	Links related logs across services
ILM	Index Lifecycle Management (ES feature)
Hot/Warm/Cold	Data tiers based on access frequency
MTTR	Mean Time To Repair (incident response)
RBAC	Role-Based Access Control
PII	Personally Identifiable Information

B. References

Documents:

- └ Architecture Diagram (attached)
- └ Data Flow Diagram (attached)
- └ Network Topology (attached)
- └ Implementation Checklist (attached)
- └ Training Materials (attached)

External Resources:

- └ Elasticsearch Docs: <https://www.elastic.co/guide/>
- └ AWS Best Practices: <https://aws.amazon.com/architecture/>
- └ GDPR Compliance: <https://gdpr-info.eu/>
- └ SOC 2 Requirements: <https://www.aicpa.org/soc>

C. Implementation Checklist

Phase 1: Infrastructure (Weeks 1-2)

- └ [] AWS account & VPC setup
- └ [] Elasticsearch cluster deployed (3 nodes)
- └ [] Logstash configured
- └ [] Kibana deployed & accessible
- └ [] Backup & disaster recovery configured
- └ [] Security audit completed
- └ [] Team training scheduled

Phase 2: E-Commerce & Mobile (Weeks 3-4)

- └ [] Filebeat installed (E-Commerce servers)
- └ [] Node.js logging configured
- └ [] Logstash pipeline for E-Commerce
- └ [] Mobile SDK integrated
- └ [] E-Commerce dashboards created (4 dashboards)
- └ [] Mobile dashboards created (4 dashboards)
- └ [] Alerting rules configured (5 rules)
- └ [] Integration tests passed (3/3)

Phase 3: Desktop & WebApp (Weeks 5-6)

- └ [] Filebeat installed (Desktop servers)
- └ [] Log4j configured
- └ [] Logstash pipeline for Desktop
- └ [] Filebeat installed (WebApp servers)
- └ [] PHP logging configured
- └ [] Logstash pipeline for WebApp
- └ [] Desktop dashboards created (3 dashboards)
- └ [] WebApp dashboards created (4 dashboards)
- └ [] Cross-project dashboards (2 dashboards)
- └ [] Integration tests passed (all projects)

Phase 4: Go-Live (Weeks 7-8)

- └ [] Performance tuning completed
- └ [] Team training completed (attendance > 90%)
- └ [] Documentation finalized
- └ [] Production hardening checklist
- └ [] 24-hour production test (100% uptime)
- └ [] Go-live announcement
- └ [] 24/7 support activated
- └ [] Celebrate success!

Document Status: DRAFT - Awaiting stakeholder review and approval

Next Steps:

1. Present to leadership (CTO, VP Product, VP Finance)
2. Collect feedback (target: 2026-05-07)
3. Revise based on feedback
4. Final approval (target: 2026-05-08)
5. Implementation kick-off (target: 2026-05-20)

Contact: DevOps Team (Daniel Foster - daniel.foster@cti-adp.com)

Document Version 1.0 | Created: 2026-05-04 | Next Review: 2026-05-11